

Post-Stall Navigation with Fixed-Wing UAVs using Onboard Vision

Adam Polevoy¹, Max Basescu¹, Luca Scheuer¹, Joseph Moore¹

Abstract—Recent research has enabled fixed-wing unmanned aerial vehicles (UAVs) to maneuver in constrained spaces through the use of direct nonlinear model predictive control (NMPC) [1]. However, this approach has been limited to a priori known maps and ground truth state measurements. In this paper, we present a direct NMPC approach that leverages NanoMap [2], a light-weight point-cloud mapping framework to generate collision-free trajectories using onboard stereo vision. We first explore our approach in simulation and demonstrate that our algorithm is sufficient to enable vision-based navigation in urban environments. We then demonstrate our approach in hardware using a 42-inch fixed-wing UAV and show that our motion planning algorithm is capable of navigating around a building using a minimalistic set of goal-points. We also show that storing a point-cloud history is important for navigating these types of constrained environments.

I. INTRODUCTION

Rotary-wing unmanned aerial vehicles (UAVs) are often preferred for navigating in constrained environments. Not only are they capable of near zero turn radii, but many are amendable to differentially flat representations that can greatly reduce computational requirements for online trajectory optimization [3]. By contrast, fixed-wing UAVs offer significant advantages over rotary-wing UAVs in terms of energy efficiency, endurance, and speed. Traditionally fixed-wing UAVs have been hindered by limited maneuverability and restricted to large open spaces. Recent research has shown that aerobatic fixed-wing UAVs are capable of navigating through complex environments by exploiting the full flight envelope [4], [1]. By operating beyond the conventional flight regimes, there is an opportunity to achieve both range and maneuverability in a single vehicle.

In [1], the authors demonstrated that a fixed-wing UAV could navigate a narrow corridor by executing post-stall turns via nonlinear model predictive control (NMPC). While this approach proved a promising means to increase fixed-wing maneuverability, it required an a priori map of the environment. For real-world applications, full a priori map information is often not practical. Rather, the vehicle must navigate in an unknown environment and construct a map during runtime for collision avoidance and navigation.

Real-time mapping poses a number of significant challenges, for aerobatic, fast-moving fixed-wing UAVs. Slower-moving multi-rotor UAVs have long relied on sensors that leverage LIDAR (e.g., [5]). However, these sensors are often too heavy for small aerobatic fixed-wings, have update rates

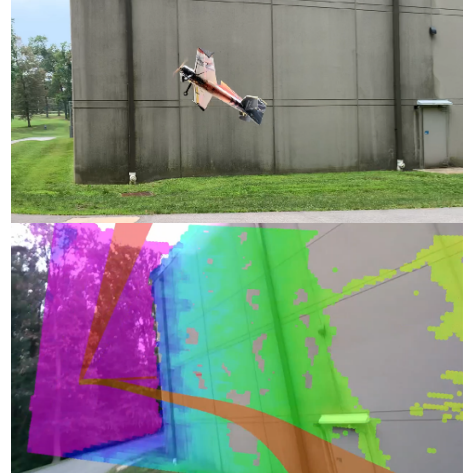


Fig. 1. Screenshot of aerobatic fixed-wing executing a post-stall maneuver (top) and corresponding point cloud data (bottom). The global path plan is shown in red and the current trajectory is shown in orange.

too slow for rapid maneuvering, have returns restricted to two-dimensional spaces, or suffer from a significantly sparse set of range measurements. With regards to mapping algorithms, commonly used occupancy-grid based techniques such as [6] have proved to be too slow and computationally intensive for use in fast flight. These challenges are further exacerbated by state and range-sensor uncertainty, which can dramatically impact map accuracy.

In this paper, we present an approach that leverages direct NMPC and an existing mapping framework, NanoMap [2], to enable a vision-based fixed-wing navigation system capable of leveraging post-stall flight for aggressive maneuvers. Our receding-horizon controller continually plans into unknown space, using distance information from the NanoMap framework in-the-loop for both nominal low-fidelity global path generation and higher-fidelity real-time trajectory optimization. The use of NanoMap allows us to reason about sensor uncertainty as well as sensor measurement history. We demonstrate our approach in hardware using a 42-inch wingspan UAV with onboard sensing and computation.

II. RELATED WORK

A majority of the research into UAV navigation and control using onboard sensing has been conducted with multirotor UAVs. Early research in this area focused on LIDAR-based simultaneous localization and mapping (SLAM), and leveraging generated maps for navigation and collision avoidance [7], [8], [5]. As the flight speeds increased, onboard vision emerged as another primary sensing solution [9], [10], [11]. Researchers also began to address the challenges associated with the computational costs of online mapping,

¹Johns Hopkins University Applied Physics Lab, Laurel, MD, 20723 {Adam.Polevoy, Max.Basescu, Luca.Scheuer, Joseph.Moore}@jhuapl.edu

DISTRIBUTION STATEMENT A: Approved for public release; distribution is unlimited.

as well as the challenges posed by large amounts of state uncertainty. [12] utilizes Gaussian Mixture Model (GMM) based mapping to reduce the memory requirements of storing and transmitting mapping data. [13] addresses the mapping challenge by maintaining a KD-tree representation of LIDAR measurements, generating a valid flight corridor, and optimizing trajectories within that corridor. Similarly, [14] utilizes a trajectory library approach on a KD-tree representation of current depth camera measurements to minimize the probability of collision. The authors then extend this work with a lightweight mapping data structure, NanoMap [2], which maintains a history of sensor measurements. For control and planning, most of approaches cited above exploit differential flatness to generate multirotor trajectories in real-time. More recent work has begun to explore NMPC for quadcopter flight, especially in the context of perception-aware control [15].

Several efforts have explored the use of onboard sensing for fixed-wing navigation. In [16], the authors demonstrated the ability to fly in a constrained-space using a scanning LIDAR. Their work focused on the state-estimation problem and used a differentially flat model valid for a reduced flight envelope. In [17], authors demonstrated fixed-wing flight using onboard stereo vision and a trajectory-library approach limited to conventional angle-of-attack regimes.

A number of researchers have also focused solely on the fixed-wing planning and control problem, assuming external sensing and known environments. These efforts have placed a greater emphasis on maneuvering across unconventional attitudes and flow regimes. Early work [18] and [19] explored post-stall flight as well as transitions to-and-from a prop-hang configuration. In [20], a library of trajectories was used to enable robust post-stall perching performance. Motion planning for fixed-wing UAVs was also explored in [21], where a library of funnels was used to navigate through a known obstacle field in real-time. [22] explored real-time trajectory motion-planning of fixed-wing UAVs using simplified models restricted to low angle-of-attack domains. In [4], [23], [24], [25], [26], researchers have developed motion planners for aerobatic fixed-wing UAVs using trajectory libraries generated offline using high fidelity physics models. In some cases, especially in the presence of high winds, NMPC has been utilized to control fixed-wing UAVs [27]. In [1], a receding-horizon NMPC approach was demonstrated that enabled fixed-wing UAVs to execute post-stall maneuvers to navigate constrained environments.

Here, we build on the work done in [1] and develop an NMPC-based navigation approach capable of running in real-time using stereo vision. Our method is distinct from other approaches in that it allows seamless exploitation of the full flight envelope for navigation via NMPC, leverages a point-cloud map to maintain a history of sensor data, and provides a means for handling uncertainty in sensor data and state estimates. Through simulation, we show the importance of compensating for sensor uncertainty, as well as the necessity of reasoning about field-of-view history, especially in the context of aggressive post-stall turns. We demonstrate our

results in hardware, and to our knowledge, this is the first demonstration of NMPC for post-stall fixed-wing flight and collision avoidance using onboard vision. We also present a framework which allows for hardware flight testing with a simulated point cloud sensor, to facilitate safe at-altitude testing in virtual environments.

III. APPROACH

In this paper, we expand upon the control strategy proposed in [1]. This control strategy consists of four major stages: RRT generation and pruning, spline-based smoothing, direct trajectory optimization, and local linear feedback. We make use of the same dynamics model for a larger 42-inch wingspan vehicle.

We assume the use of a mounted depth camera to perform sensing, and light-weight mapping is performed with NanoMap. RRT paths are reused whenever possible to reduce computational load and to maintain consistency between control iterations. Horizon point selection is constrained to known regions of the map to prevent trajectory generation through unobserved obstacles. Constraints on the distance to obstacles and estimated probability of collision are both evaluated for trajectory generation.

A. Mapping

NanoMap was chosen in place of OctoMap because it has a low map maintenance cost and is less likely to propagate odometry error due to its local map structure. It is also able to propagate state uncertainty through a local history of depth measurements.

A few adjustments were made to NanoMap to better suit our control strategy. Query replies were modified to include the transformation from the query body frame to the current body frame. This transformation is necessary for calculating the obstacle constraint gradients.

NanoMap query replies include information about the field of view status of the query point. This information specifies if the query point is in free space, occluded space, or outside of the field of view of depth measurements. If the query point was not within the field of view free space of any prior depth measurements, it had no valid corresponding depth measurement. In this case, the obstacle points are pulled from the most recent depth measurement. We modified NanoMap to return the K-nearest obstacle points from the most recent occluded field of view instead. This yielded more accurate nearest obstacle points, especially while planning through occluded space.

B. RRT Generation

In the base controller proposed in [1], a rapidly-exploring random tree (RRT) was used to generate global paths to the goal for horizon point selection. The RRT was expanded until the goal point was connected to the tree, and the path from the current position to the goal position was extracted. This path was iteratively pruned, smoothed with Bézier curves, and parameterized in terms of time. We refer to this final

smoothed path as the smoothed RRT path, and it is used to select a receding horizon point for trajectory generation.

This strategy poses a problem for planning in unknown environments; the horizon point may be in or behind an unobserved obstacle. This is undesirable since the direct trajectory optimization problem is warm started with the previously generated trajectory. If a previously generated trajectory becomes intersected by newly obtained map data, the optimizer will often be unable to find a feasible solution.

To avoid this, the RRT path is constrained to known regions of the map using a method similar to that proposed in [28]. Our method is outlined in Algorithm 1. As the tree expands, a list of frontier nodes (which exist in unknown space and connect to nodes in known space) is maintained. A node is in a known region of the map if the NanoMap query returns a field of view status indicating that the query point is in free space. If the goal point is not connected to the tree, or if the goal point is in an unknown region of the map, then the frontier node closest to the goal position is used as the goal instead.

Algorithm 1: RRT($x_{init}, x_{goal}, \Delta x, \text{map}, K, \Delta t$)

```

 $\tau.\text{init}(x_{init}), f \leftarrow \emptyset$ 
for  $k = 1$  to  $K$  do
   $x_{rand} \leftarrow \text{RANDOM\_STATE}()$ 
   $x_{near} \leftarrow \text{NEAREST\_NEIGHBOR}(x_{rand}, \tau)$ 
   $u \leftarrow \text{SELECT\_INPUT}(x_{rand}, x_{near})$ 
   $x_{new} \leftarrow \text{NEW\_STATE}(x_{near}, u, \Delta t)$ 
  if  $\text{PATH\_OBST\_FREE}(x_{near}, x_{new}, \text{map})$  then
     $\tau.\text{add\_vertex}(x_{new})$ 
     $\tau.\text{add\_edge}(x_{near}, x_{new}, u)$ 
    if  $\text{DISTANCE}(x_{new}, x_{goal}) \leq \Delta x$  then
       $x_{goal} \leftarrow x_{new}$ 
      break
    if  $\text{IS\_FRONTIER\_NODE}()x_{new}, \text{map})$  then
       $f.\text{add}(x_{new})$ 
if  $x_{goal}$  not in  $\tau$  or  $\text{IN\_UNKNOWN}(x_{goal}, \text{map})$  then
   $x_{goal} \leftarrow x_f \leftarrow x_{goal}.\text{FIND\_CLOSEST}(f)$ 
Return  $\tau, x_{goal}$ 

```

At each control iteration, a truncated version of the prior RRT path is used to initialize the RRT. This truncation takes into consideration both the updated vehicle state and any newly detected obstacles. The beginning of the RRT path is truncated to minimize the distance between the start of the path and the vehicle state; the end is truncated to ensure that the path does not intersect with obstacles.

Reuse of the prior path reduces RRT computation time and also provides a more gradual evolution of the RRT path. This leads to a more incremental update of the receding horizon goal point and a reduced the computational burden for the warm-started trajectory optimization routine.

C. Direct Trajectory Optimization

While the base direct trajectory optimization problem is the same as in [1], NanoMap queries are used to calculate

the obstacle avoidance constraints instead of Octomap.

Our optimization problem can be written as

$$\begin{aligned}
 \min_{\mathbf{x}_k, \mathbf{u}_k, h} \quad & \sum_{k=0}^N [\mathbf{x}_k^T \mathbf{Q} \mathbf{x}_k] + \mathbf{x}_{N+1}^T \mathbf{Q}_f \mathbf{x}_{N+1} \\
 \text{s.t.} \quad & \forall k \in [0, \dots, N] \text{ and} \\
 & \mathbf{x}_k - \mathbf{x}_{k+1} + \frac{h}{6.0} (\dot{\mathbf{x}}_k + 4\dot{\mathbf{x}}_{c,k} + \dot{\mathbf{x}}_{k+1}) = 0 \\
 & \mathbf{x}_f - \delta_f \leq \mathbf{x}_N \leq \mathbf{x}_f + \delta_f \\
 & \mathbf{x}_i - \delta_i \leq \mathbf{x}_N \leq \mathbf{x}_i + \delta_i \\
 & \mathbf{x}_{min} \leq \mathbf{x}_k \leq \mathbf{x}_{max}, \quad \mathbf{u}_{min} \leq \mathbf{u}_k \leq \mathbf{u}_{max} \\
 & c(\mathbf{x}) \geq 0 \\
 & h_{min} \leq h \leq h_{max}
 \end{aligned} \tag{1}$$

where

$$\begin{aligned}
 \dot{\mathbf{x}}_k &= \mathbf{f}(t, \mathbf{x}_k, \mathbf{u}_k), \quad \dot{\mathbf{x}}_{k+1} = \mathbf{f}(t, \mathbf{x}_{k+1}, \mathbf{u}_{k+1}) \\
 \mathbf{u}_{c,k} &= (\mathbf{u}_k + \mathbf{u}_{k+1})/2 \\
 \mathbf{x}_{c,k} &= (\mathbf{x}_k + \mathbf{x}_{k+1})/2 + h(\dot{\mathbf{x}}_k - \dot{\mathbf{x}}_{k+1})/8 \\
 \dot{\mathbf{x}}_{c,k} &= \mathbf{f}(t, \mathbf{x}_{c,k}, \mathbf{u}_{c,k}).
 \end{aligned} \tag{2}$$

Here, $\mathbf{x}$ signifies the system state, $\mathbf{u}$ is the control actions, and $\mathbf{f}(t, \mathbf{x}_{k+1}, \mathbf{u}_{k+1})$ is the dynamical model. h is the time step bounded from $h_{min} = 0.001s$ to $h_{max} = 0.2s$, δ_f and δ_i represent the bounds on the desired final and initial states ($\mathbf{x}_f, \mathbf{x}_i$), respectively. N is the number of knot points, and $c(\mathbf{x})$ represents the collision constraint.

The first constraint that we evaluated was $c(\mathbf{x}) = d(\mathbf{x}) - r$, or the distance to obstacles, where r is the obstacle radius. Given the query, $\mathbf{x}$, and the $K = 10$ nearest neighbors $\mathbf{a}_i$, the distance to each nearest neighbor is calculated as $d(\mathbf{x})_i = \|\mathbf{x} - \mathbf{a}_i\|$. The distance to obstacle constraint is calculated as the average distance to the nearest neighbors $d(\mathbf{x}) = \frac{1}{K} \sum_{i=1}^K (d(\mathbf{x})_i)$. Similarly, the gradient for this constraint is calculated as $\nabla d(\mathbf{x}) = \frac{1}{K} \sum_{i=1}^K \left(\frac{\mathbf{x} - \mathbf{a}_i}{\|\mathbf{x} - \mathbf{a}_i\|} \right)$.

The second constraint we evaluated was the probability of collision; we replaced $d(\mathbf{x}) \geq r$ with $p(\mathbf{x}) \leq s$, where s is the maximum probability of collision. Probability of collision is calculated using the formulation presented in [29]. Specifically, the probability between a query point and a nearest neighbor is calculated as

$$\begin{aligned}
 p_i(\mathbf{x}) &= \sum_{k=0}^{\infty} (-1)^k c_k \frac{(r+s)^{2\frac{n}{2}+k}}{\Gamma(\frac{n}{2}+k+1)} \\
 c_0 &= \exp \left(-\frac{1}{2} \sum_{j=1}^n b_{ij}^2 \right) \prod_{j=1}^n (2\lambda_j)^{\frac{1}{2}} \\
 c_k &= \frac{1}{k!} \sum_{j=0}^{k-1} d_{(k-j)} c_j \\
 d_k &= \frac{1}{2} \sum_{j=1}^n (1 - k b_{ij}^2) (2\lambda_j)^{-k} \\
 \mathbf{b}_i &= \Sigma^{-\frac{1}{2}} (\mathbf{x} - \mathbf{a}_i) \\
 \lambda &= \text{eigenvalues}(\Sigma)
 \end{aligned} \tag{3}$$

where $n = 3$ is the number of spatial dimensions and Σ is the sum of the covariance of the query point and of the nearest neighbor. The probability of collision between a query point and all nearest neighbors is calculated as in [2]:

$$p(\mathbf{x}) = 1 - \prod_{i=1}^K [1 - p_i] \quad (4)$$

The gradient for this constraint is calculated as:

$$\begin{aligned} \nabla p(\mathbf{x}) &= (1 - p(\mathbf{x})) \sum_{i=1}^K \left[\frac{\nabla p_i(\mathbf{x})}{1 - p_i(\mathbf{x})} \right] \\ \nabla p_i(\mathbf{x}) &= \sum_{k=0}^{\infty} \left[(-1)^k \nabla c_k \frac{(r+s)^{2\frac{n}{2}+k}}{\Gamma(\frac{n}{2}+k+1)} \right] \\ \nabla c_0 &= c_0 \cdot \Sigma^{-\frac{1}{2}} \mathbf{b}_i \\ \nabla c_k &= \frac{1}{k} \sum_{j=0}^{k-1} [d_{k-j} \cdot \nabla c_j + \nabla d_{k-j} \cdot c_j] \\ \nabla d_k &= -\frac{1}{2} k (2\lambda)^{-kT} \Sigma^{-\frac{1}{2}} \mathbf{b}_i \end{aligned} \quad (5)$$

We significantly sped up these probability estimation calculations through the use of bottom up dynamic programming for the d and c parameters for each query. We found that the summing a maximum of 75 terms for $p_i(\mathbf{x})$ was sufficient. Additionally, the assumption of a diagonal covariance matrix significantly speeds up the calculation of $\Sigma^{-\frac{1}{2}}$ and λ .

IV. REAL-TIME SIMULATION STUDY

A. Simulation Setup

We evaluated the effectiveness of our approach in an unknown map through simulation experiments. Each experiment consisted of 100 trials. A trial was considered successful if the fixed-wing navigated around the hallway without collision.

The dynamics simulator and the controller consist of custom C++ ROS nodes. The depth camera is simulated using Gazebo at 30Hz with a resolution of 128x85, field of view of 87x58 degrees, and a 20 meter sensing range. The dynamics simulator provides the Gazebo simulation with state information for the camera and generates the simulated point cloud data input to the controller. The camera is rigidly attached to the fixed-wing behind the propeller.

The map is a tight hallway with two 90 degree turns, which showcases the aerobatic post-stall maneuvers the system is required to make around buildings in urban environments. Additionally, the 90 degree turns obstruct the camera field of view.

B. Experiment 1: Use of NanoMap History

The first set of simulation experiments showcases the necessity of reasoning about field-of-view history. The controller was run in simulation without any prior knowledge of the map, but with perfect state information and noiseless depth data. The controller was run using the distance to obstacles constraint with a minimum distance constraint of

1.2 meters. In this experiment, a cost was placed on the final state of the trajectories in lieu of a hard constraint.

NanoMap with a history of 50 point clouds was used. The system was successful in navigation to goal for 91 trials, of which only 15.4% broke the obstacle constraint, as shown in Figure 2a.

To highlight the trajectory optimizer's use of the NanoMap measurement history in these trials, we tracked the number of queries that occurred at a point cloud history depth for each iteration during the trial. Figure 3 shows the NanoMap search depth during direct transcription across all trajectory generation iterations. Figure 4 shows the NanoMap search depth during one iteration as the fixed-wing turns a corner. For these data, it is clear that the optimizer makes use of NanoMap's point cloud history as it performs aerobatic maneuvers that require the fixed-wing to quickly turn outside of its field of view.

Next, we performed the same simulation using only the most recent point cloud measurement instead of NanoMap's history. When doing so, the system was only successful 58 times, of which 57.4% broke the obstacle constraint (Figure 2c). The failure case for the majority of these trials was due to the RRT global path planning failure.

Thus, to evaluate the effect of using NanoMap on trajectory generation in particular, we allowed the RRT planner to access NanoMap, but restricted the direct transcription trajectory generator to the most recent point cloud. The system was only successful in reaching the goal without collision 80 times, of which 22.5% broke the obstacle constraint (Figure 2b).

C. Experiment 3: Noise

The second set of simulation experiments showcases the importance of compensating for sensor uncertainty. Positional noise, $\mathcal{N}(\text{diag}(0, 0, 0), \text{diag}(0.1, 0.1, 0.1))$, was injected into the fixed-wing state to simulate state estimation noise. Noise was also injected into the depth images, $\mathcal{N}(0, 0.316)$, to simulate depth camera noise. To evaluate the effects of noise only on trajectory generation, a global path to goal is provided in lieu of the RRT planner.

Three different constraints were evaluated; distance to obstacles, distance to obstacles with standard deviation inflation, and probability of collision. For the probability of collision constraint, we set a max probability of collision of 0.5 and a robot radius of 1.2 meters. For the distance to obstacles with standard deviation inflation constraint, we inflate the collision radius using the standard deviation values.

The distance to obstacle constraint had 99 successful trials, of which only 17.2% broke the obstacle constraint (Figure 2d). The distance to obstacles with standard deviation inflation trials had 95 successful trials, of which only 1% broke the obstacle constraint (Figure 2e). The probability of collision constraint had 100 successful trials, of which only 1% broke the obstacle constraint (Figure 2f). Thus, we can conclude that explicitly taking the covariance of our state and noise into account benefits planning.

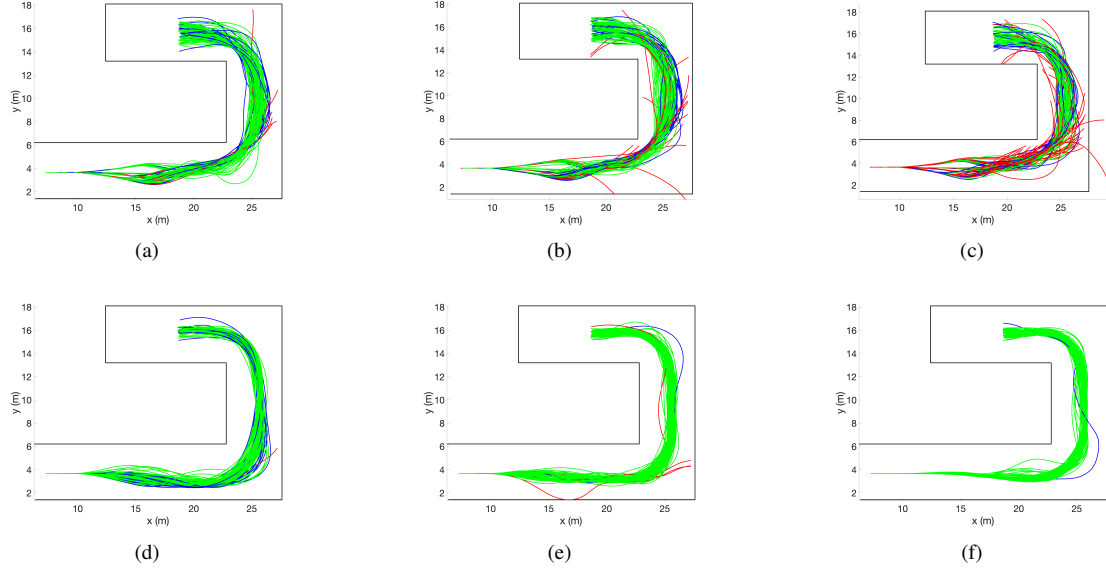


Fig. 2. Successful trials in green, trials without collision but with constraint violation in blue, trials with collisions in red. (a) Trials using full NanoMap history. (b) Only direct transcription restricted to most recent point cloud. (c) RRT and direct transcription restricted to most recent point cloud. (d) Distance to obstacle constraint. (e) Distance to obstacle constraint using standard deviation inflation. (f) Probability of collision constraint. (d), (e), and (f) include measurement and state estimation noise.

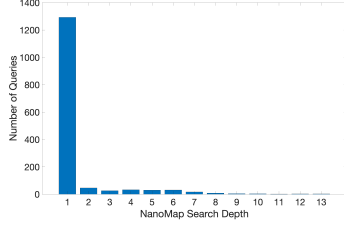


Fig. 3. NanoMap search depth queries across all iterations for the trajectory optimizer.

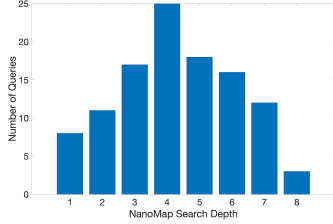


Fig. 4. NanoMap search depth queries during optimization of a single trajectory while turning a corner.

V. HARDWARE EXPERIMENTS

A. Experimental Set-up

To evaluate our method on physical hardware, we used an EPP Edge 540 XL from Twisted Hobbies. State estimation was provided by an mRobotics Control Zero F7 flight controller running Ardupilot firmware. Additional external sensors including a uBlox ZED-F9P RTK GPS receiver and an MS5525 airspeed sensor were used to improve the flight controller’s state estimation. An RFD900u telemetry radio was used to monitor the aircraft in flight as well as provide RTK corrections to the GPS receiver. A USB to Serial adapter passed state data from the flight controller to an

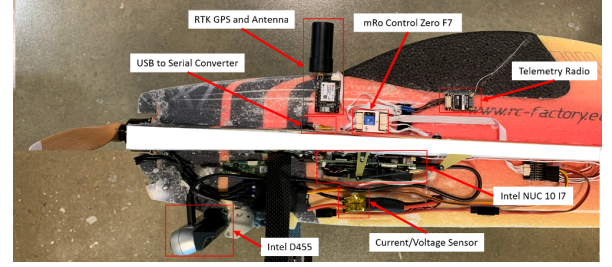


Fig. 5. A close-up view of the Edge540XL vehicle and the key components for onboard sensing and control.

Intel NUC 10 i7 where our control algorithm ran. An Intel Realsense D455 camera was rigidly attached to the fuselage behind the propeller. Figure 5 shows an image of the plane with labeled components.

The mission for the UAV was to fly towards two way-points, both of which were obstructed by a large building. This environment showcases both straight, unobstructed zones as well as hard obstacles. Navigation to the waypoints requires the system to make tight turns between the building and trees, which is similar to our hallway simulation environment. A mission was considered successful if the fixed-wing was able to navigate around the building.

B. Simulated Perception

To evaluate our method using perfect perception data, we first simulated the depth camera data using Gazebo onboard the plane during a physical test flight. To do this, we provided the Gazebo simulation with an accurate mesh of the building and nearby light poles. As the fixed-wing flew around the building, it sent odometry measurements to the simulator. In return, it received noiseless point cloud data given the camera field of view. The fixed-wing used the distance to obstacles

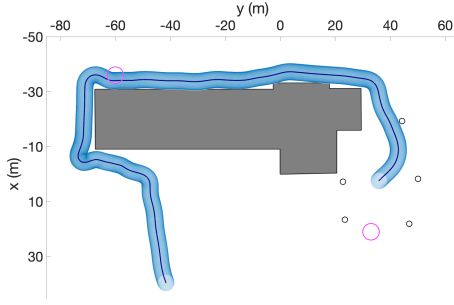


Fig. 6. Plot shows the flight path taken by the Edge540XL vehicle around a physical building using a simulated depth camera for navigation. The gray shaded region represents the building and the black circles represent the light poles.

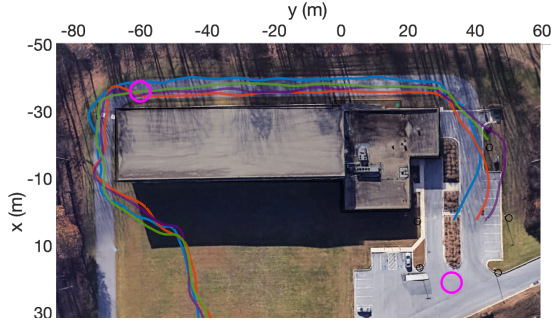


Fig. 7. An overhead view of four flight paths taken by the Edge540XL vehicle around a physical building using the onboard RealSense D455 stereo depth camera for navigation. The objective of the controller is to navigate, without collision, to the two pink circle goal regions in sequence.

with standard deviation inflation with a 3 meter radius, a time horizon $T_H = 1s$ and a replanning frequency of 5Hz.

The fixed-wing was successfully able to navigate around the building to the waypoints using the simulated depth camera data. Figure 6 shows the fixed-wing's flight path overlaid with the obstacle distance constraint radius.

C. Control Experiment

After confirming that our method worked on hardware with perfect perception data, we ran the same experiment with the same controller parameters using the depth data from the mounted D455. The depth images from the camera were down sampled by a factor of 10 and converted to point clouds for NanoMap.

Out of six hardware trials, four successfully rounded the building. In one of these trials, the fixed-wing crashed into a light pole after making it around the building. This crash was caused by the inability of the depth sensor to see the pole until the fixed-wing was too close to react.

The paths of the successful trials are shown in Figure 7, and the obstacle distance constraint radii are overlaid in Figure 8. Additionally, the search depths of NanoMap queries during a turn are shown in Figure 9.

Two of the hardware trials failed, crashing into the building shortly after executing a turning maneuver (see Figure 10). In both of these failure cases, the fixed-wing turned towards the building very late after rounding the corner. When this happens, the depth camera does not observe the

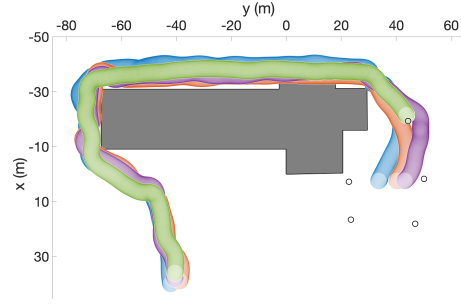


Fig. 8. An overhead view of the four flight paths taken by the Edge540XL vehicle with the desired collision radius superimposed to highlight obstacle avoidance performance.

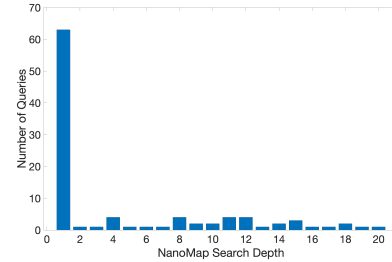


Fig. 9. NanoMap search depth queries for a hardware post-stall turn.

part of the wall after the corner, RRT paths are generated through the apparent gap, and the vehicle crashes.

VI. DISCUSSION

In this work, we outlined a controller capable of enabling agile fixed-wing flight in unknown environments. In simulation, we showed the ability to navigate a tight hallway environment with a fixed-wing UAV. We also demonstrated the ability of a fixed-wing to avoid and navigate around a building without prior knowledge of its environment. In the future, we plan to explore improving system robustness by more tightly coupling perception and control. For instance, by augmenting the optimizer's cost function, we may be able to reduce failure cases by rewarding information gathering during aggressive maneuvers. We may also be able to improve performance by using generative adversarial networks, as was done in [30], to predict and reason about the environment beyond the field of view.

VII. ACKNOWLEDGEMENT

This material is based upon work supported by the DARPA OFFSET program. The views, opinions and/or findings expressed are those of the author and should not be interpreted as representing the official views or policies of the Department of Defense or the U.S. Government.

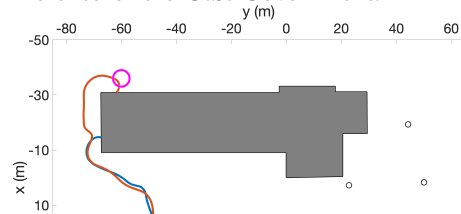


Fig. 10. Plot shows the flight paths of the failed hardware trials.

REFERENCES

- [1] M. Basescu and J. Moore, "Direct nmppc for post-stall motion planning with fixed-wing uavs," *arXiv preprint arXiv:2001.11478*, 2020.
- [2] P. R. Florence, J. Carter, J. Ware, and R. Tedrake, "Nanomap: Fast, uncertainty-aware proximity queries with lazy search over local 3d data," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2018, pp. 7631–7638.
- [3] D. Mellinger and V. Kumar, "Minimum snap trajectory generation and control for quadrotors," in *2011 IEEE international conference on robotics and automation*. IEEE, 2011, pp. 2520–2525.
- [4] J. M. Levin, A. Paranjape, and M. Nahon, "Agile fixed-wing uav motion planning with knife-edge maneuvers," in *2017 International Conference on Unmanned Aircraft Systems*. IEEE, 2017, pp. 114–123.
- [5] S. Grzonka, G. Grisetti, and W. Burgard, "Towards a navigation system for autonomous indoor flying," in *2009 IEEE International Conference on Robotics and Automation*. IEEE, 2009, pp. 2878–2883.
- [6] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, "Octomap: An efficient probabilistic 3d mapping framework based on octrees," *Autonomous Robots*, vol. 34, no. 3, pp. 189–206, 2013.
- [7] A. Bachrach, S. Prentice, R. He, and N. Roy, "Range-robust autonomous navigation in gps-denied environments," *Journal of Field Robotics*, vol. 28, no. 5, pp. 644–666, 2011.
- [8] S. Shen, N. Michael, and V. Kumar, "Autonomous multi-floor indoor navigation with a computationally constrained mav," in *2011 IEEE International Conference on Robotics and Automation*. IEEE, 2011, pp. 20–25.
- [9] M. Blösch, S. Weiss, D. Scaramuzza, and R. Siegwart, "Vision based mav navigation in unknown and unstructured environments," in *2010 IEEE International Conference on Robotics and Automation*. IEEE, 2010, pp. 21–28.
- [10] M. Faessler, F. Fontana, C. Forster, E. Mueggler, M. Pizzoli, and D. Scaramuzza, "Autonomous, vision-based flight and live dense 3d mapping with a quadrotor micro aerial vehicle," *Journal of Field Robotics*, vol. 33, no. 4, pp. 431–450, 2016.
- [11] A. S. Huang, A. Bachrach, P. Henry, M. Krainin, D. Maturana, D. Fox, and N. Roy, "Visual odometry and mapping for autonomous flight using an rgb-d camera," in *Robotics Research*. Springer, 2017, pp. 235–252.
- [12] W. Tabib and N. Michael, "Simultaneous localization and mapping of subterranean voids with gaussian mixture models," in *Field and Service Robotics*. Springer, 2021, pp. 173–187.
- [13] F. Gao, W. Wu, W. Gao, and S. Shen, "Flying on point clouds: Online trajectory generation and autonomous navigation for quadrotors in cluttered environments," *Journal of Field Robotics*, vol. 36, no. 4, pp. 710–733, 2019.
- [14] P. Florence, J. Carter, and R. Tedrake, "Integrated perception and control at high speed: Evaluating collision avoidance maneuvers without maps," in *Algorithmic Foundations of Robotics XII*. Springer, 2020, pp. 304–319.
- [15] D. Falanga, P. Foehn, P. Lu, and D. Scaramuzza, "Pampc: Perception-aware model predictive control for quadrotors," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 1–8.
- [16] A. Bry, C. Richter, A. Bachrach, and N. Roy, "Aggressive flight of fixed-wing and quadrotor aircraft in dense indoor environments," *The International Journal of Robotics Research*, vol. 34, no. 7, pp. 969–1002, 2015.
- [17] A. J. Barry, P. R. Florence, and R. Tedrake, "High-speed autonomous obstacle avoidance with pushbroom stereo," *Journal of Field Robotics*, vol. 35, no. 1, pp. 52–68, 2018.
- [18] R. Cory and R. Tedrake, "Experiments in fixed-wing uav perching," in *AIAA Guidance, Navigation and Control Conference and Exhibit*, 2008, p. 7256.
- [19] F. Sobic and J. How, "Nonlinear agile control test bed for a fixed wing aircraft in a constrained environment," in *AIAA Infotech@Aerospace Conference and AIAA Unmanned... Unlimited Conference*, 2009, p. 1927.
- [20] J. Moore, R. Cory, and R. Tedrake, "Robust post-stall perching with a simple fixed-wing glider using lqr-trees," *Bioinspiration & biomimetics*, vol. 9, no. 2, p. 025013, 2014.
- [21] A. Majumdar and R. Tedrake, "Funnel libraries for real-time robust feedback motion planning," *The International Journal of Robotics Research*, vol. 36, no. 8, pp. 947–982, 2017.
- [22] H. Alturbeh and J. F. Whidborne, "Real-time obstacle collision avoidance for fixed wing aircraft using b-splines," in *Control (CONTROL), 2014 UKACC International Conference on*. IEEE, 2014, pp. 115–120.
- [23] W. Khan and M. Nahon, "Modeling dynamics of agile fixed-wing uavs for real-time applications," in *2016 international conference on unmanned aircraft systems (ICUAS)*. IEEE, 2016, pp. 1303–1312.
- [24] E. Bulka and M. Nahon, "Autonomous fixed-wing aerobatics: from theory to flight," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2018, pp. 6573–6580.
- [25] J. M. Levin, M. Nahon, and A. A. Paranjape, "Real-time motion planning with a fixed-wing uav using an agile maneuver space," *Autonomous Robots*, pp. 1–20, 2019.
- [26] E. Bulka and M. Nahon, "High-speed obstacle-avoidance with agile fixed-wing aircraft," in *2019 International Conference on Unmanned Aircraft Systems (ICUAS)*. IEEE, 2019, pp. 971–980.
- [27] S. Mathisen, K. Gryte, S. Gros, and T. A. Johansen, "Precision deep-stall landing of fixed-wing uavs using nonlinear model predictive control," *Journal of Intelligent and Robotics Systems*, vol. 101, no. 24, 2020.
- [28] H. Umari and S. Mukhopadhyay, "Autonomous robotic exploration based on multiple rapidly-exploring randomized trees," in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2017, pp. 1396–1402.
- [29] A. Thomas, F. Mastrogianni, and M. Baglietto, "An integrated localization, motion planning and obstacle avoidance algorithm in belief space," *Intelligent Service Robotics*, vol. 14, no. 2, pp. 235–250, 2021.
- [30] K. D. Katyal, A. Polevoy, J. Moore, C. Knuth, and K. M. Popek, "High-speed robot navigation using predicted occupancy maps," *arXiv preprint arXiv:2012.12142*, 2020.